

The Codex

Packages, Environments and the Alchemist's Toolchain

Summary: Set up a sealed laboratory, stock it with reagents, guard its secrets, then organise your formulas into a real Python package: virtual environments, dependency management, configuration through the environment, and every import mystery from package initialisation to circular dependencies.

Version: 1.0

Contents

I	**Introduction**	**2**
II	**AI Instructions**	**3**
III	**General Instructions**	**5**
IV	**Exercise 0: The Sealed Laboratory**	**7**
V	**Exercise 1: The Oracle**	**11**
VI	**Exercise 2: The Alembic**	**14**
VII	**Exercise 3: Distillation and Transmutation**	**16**
VIII	**Exercise 4: Avoid the Explosion**	**19**
IX	**Submission Instructions**	**22**

Chapter I

Introduction

Welcome to the Alchemist's Laboratory, young apprentice!

In medieval times, alchemists sought to transform base metals into gold. Today, as a Python alchemist, you will learn to transform scattered code into organised, reusable magical formulas. Just as ancient alchemists had their grimoires organised into chapters and sections, Python code must be organised into modules and packages.

But no alchemist starts with the grimoire. First comes the laboratory itself: a workspace sealed off from the outside world, so that a failed experiment never contaminates anything beyond its own walls. Then come the reagents you bring in from elsewhere, and the sigils that keep your secrets out of the pages anyone can read.

Only then can you write the formulas. And that is where the real mysteries begin:

- Using `__init__.py`, the gateway that transforms a simple folder into a magical package
- Choosing precisely what your package exposes, and what stays hidden
- Calling the code of a distant module from a distant module
- Managing absolute vs relative imports
- Avoiding circular dependencies

In this codex you will build the whole laboratory, in order: the sealed workspace, the reagent supply, the secret sigils, and finally the grimoire itself.



Focus on understanding **how these tools work**, not just on making them work. A true alchemist understands every step of the transmutation process!

Chapter II

AI Instructions

Context

During your learning journey, AI can assist with many different tasks. Take the time to explore the various capabilities of AI tools and how they can support your work. However, always approach them with caution and critically assess the results. Whether it's code, documentation, ideas, or technical explanations, you can never be completely sure that your question was well-formed or that the generated content is accurate. Your peers are a valuable resource to help you avoid mistakes and blind spots.

Main message

- Use AI to reduce repetitive or tedious tasks.
- Develop prompting skills — both coding and non-coding — that will benefit your future career.
- Learn how AI systems work to better anticipate and avoid common risks, biases, and ethical issues.
- Continue building both technical and power skills by working with your peers.
- Only use AI-generated content that you fully understand and can take responsibility for.

Learner rules:

- You should take the time to explore AI tools and understand how they work, so you can use them ethically and reduce potential biases.
- You should reflect on your problem before prompting — this helps you write clearer, more detailed, and more relevant prompts using accurate vocabulary.
- You should develop the habit of systematically checking, reviewing, questioning, and testing anything generated by AI.
- You should always seek peer review — don't rely solely on your own validation.

Phase outcomes:

- Develop both general-purpose and domain-specific prompting skills.
- Boost your productivity with effective use of AI tools.
- Continue strengthening computational thinking, problem-solving, adaptability, and collaboration.

Comments and examples:

- You'll regularly encounter situations — exams, evaluations, and more — where you must demonstrate real understanding. Be prepared, keep building both your technical and interpersonal skills.
- Explaining your reasoning and debating with peers often reveals gaps in your understanding. Make peer learning a priority.
- AI tools often lack your specific context and tend to provide generic responses. Your peers, who share your environment, can offer more relevant and accurate insights.
- Where AI tends to generate the most likely answer, your peers can provide alternative perspectives and valuable nuance. Rely on them as a quality checkpoint.

✓ Good practice:

> I ask AI: "How do I test a sorting function?" It gives me a few ideas. I try them out and review the results with a peer. We refine the approach together.

✗ Bad practice:

> I ask AI to write a whole function, copy-paste it into my project. During peer-evaluation, I can't explain what it does or why. I lose credibility — and I fail my project.

✓ Good practice:

> I use AI to help design a parser. Then I walk through the logic with a peer. We catch two bugs and rewrite it together — better, cleaner, and fully understood.

✗ Bad practice:

> I let Copilot generate my code for a key part of my project. It compiles, but I can't explain how it handles pipes. During the evaluation, I fail to justify and I fail my project.

Chapter III

General Instructions

- Your project must be written in **Python 3.10 or later**.
- Your project must adhere to the **flake8** coding standard.
- All code must include comprehensive type annotations. Check this using `mypy` .
- All standard classes and collections are authorized, along with their methods (`int`, `str`, `list`, `dict`, etc.).
- All built-in functions are authorized, except for `eval()` and `exec()` .
- Each exercise declares its own authorized imports. Respect them: they are part of the exercise.
- It is forbidden to modify `sys.path` anywhere in this project.



Exercises 0 and 1 install and use **external libraries**, because that is exactly what they teach. Exercises 2 to 4 are the opposite: there, the **only** imports allowed are the modules and files you create yourself. Read the header of each exercise.



In exercises 2 to 4, all functions should be simple and return strings. Focus on the import concepts, not on complex logic.

This project builds one laboratory across five exercises. Exercises 0 and 1 are self-contained in their own folder. Exercises 2, 3 and 4 grow a single package tree at the root of the repository, each one adding to what the previous one built. At the end of the project, your tree should look like this:

```
.
|-- ex0
|   |-- construct.py
|   |-- loading.py
|   |-- pyproject.toml
|   `-- requirements.txt
|-- ex1
|   |-- .env.example
|   |-- .gitignore
|   |-- oracle.py
|   `-- requirements.txt
|-- alchemy
|   |-- __init__.py
|   |-- elements.py
|   |-- potions.py
|   |-- transmutation
|   |   |-- __init__.py
|   |   `-- recipes.py
|   `-- grimoire
|       |-- __init__.py
|       |-- dark_spellbook.py
|       |-- dark_validator.py
|       |-- light_spellbook.py
|       `-- light_validator.py
|-- elements.py
|-- ft_alembic_0.py
|-- ft_alembic_1.py
|-- ft_distillation_0.py
|-- ft_transmutation_0.py
|-- ft_kaboom_0.py
`-- ft_kaboom_1.py
```

Chapter IV

Exercise 0: The Sealed Laboratory

	Exercise0
ex0	
Directory: *ex0/*	
Files to Submit: `construct.py, loading.py, requirements.txt, pyproject.toml`	
Authorized: `os, sys, site, importlib, pandas, numpy`	



"What is a laboratory, if not a room the outside world cannot reach?"

Before an apprentice is allowed to mix anything, they must seal the laboratory: an isolated space where a botched experiment ruins only the experiment. In Python, that sealed room is a **virtual environment**. Once it is sealed, the apprentice may finally bring reagents in from the outside world without poisoning everything else they own; that is **dependency management**.

This first exercise covers both halves of that preparation, because neither is useful alone.

Part 1: Sealing the room

Create a program called `ex0/construct.py` that:

- Detects whether it is running inside a virtual environment
- Displays information about the current Python environment
- Provides instructions for creating and activating a virtual environment if none is detected
- Shows the difference between global and virtual environment package locations



Your program must work both inside and outside a virtual environment, and produce a clearly different output in each case.

Testing outside a virtual environment:

```
$> python3 ex0/construct.py
# Should detect no virtual environment and provide instructions
```

Creating and testing a virtual environment:

```
$> python3 -m venv lab_env
$> source lab_env/bin/activate
(lab_env) $> python3 ex0/construct.py
# Should detect the virtual environment and show details
```



Do not submit your virtual environment in your repository. You must be able to create a new one during the review if needed.

Expected output, outside the sealed laboratory:

```
$> python3 ex0/construct.py

LABORATORY STATUS: You are working in the open

Current Python: /usr/bin/python3.11
Virtual Environment: None detected

WARNING: You are in the global environment!
Every reagent you install here leaks into the whole system.

To seal the laboratory, run:
python3 -m venv lab_env
source lab_env/bin/activate # On Unix
lab_env\Scripts\activate  # On Windows

Then run this program again.
```

And inside the sealed laboratory:

```
$> python3 ex0/construct.py

LABORATORY STATUS: The laboratory is sealed

Current Python: /path/to/lab_env/bin/python
Virtual Environment: lab_env
Environment Path: /path/to/lab_env

SUCCESS: You are working in an isolated environment!
Safe to install reagents without affecting
the global system.

Reagent installation path:
/path/to/lab_env/lib/python3.11/site-packages
```

Part 2: Stocking the reagents



"I need reagents. Lots of reagents."

A sealed laboratory is useless while it is empty. Now bring in the reagents, and declare them properly so that any other alchemist can reproduce your laboratory exactly. You will do this twice, with the two tools of the trade: **pip** and **Poetry**.

Create a program called `ex0/loading.py` that:

- Declares a couple of external **reagents** (libraries such as `pandas` and `numpy`) in both `ex0/requirements.txt` (for pip) and `ex0/pyproject.toml` (for Poetry).
- Imports each declared reagent and reports its installed version.
- Shows, in its output, the difference between pip and Poetry dependency management.



Your program must detect which reagents are available and, if one is missing, print a helpful message with the install command for both pip and Poetry. It must not crash.

Testing without the reagents installed:

```
$> python3 ex0/loading.py
# Should show missing dependencies and installation instructions for pip and Poetry
```

Installing with pip:

```
$> pip install -r ex0/requirements.txt
$> python3 ex0/loading.py
# Should list the installed reagent versions
```

Installing with Poetry:

```
$> poetry install
$> poetry run python ex0/loading.py
# Should list the same versions inside the Poetry environment
```

Expected behaviour:

```
$> python3 ex0/loading.py

REAGENT STATUS: Loading reagents...

Checking dependencies:
```

```
[OK] pandas (2.1.0) - Data manipulation ready
[OK] numpy (1.25.0) - Numerical computation ready

pip vs Poetry:
  requirements.txt declares what to install; pip resolves it at install time.
  pyproject.toml declares the same constraints, and Poetry pins the resolved
  versions in poetry.lock so every install is identical.

All reagents present and accounted for.
```



What does a virtual environment actually isolate, and what does it not isolate? And why would you ever choose Poetry over a plain requirements.txt ?

Chapter V

Exercise 1: The Oracle

	Exercise1
ex1	
Directory: *ex1/*	
Files to Submit: `oracle.py, requirements.txt, .env.example, .gitignore`	
Authorized: `os, sys, python-dotenv, file operations`	



"Know thyself." And more importantly: know what your grimoire must never say out loud.

The Oracle knows everything, because the Oracle can read the configuration of the whole laboratory. In the real world, programs need access to sensitive knowledge: database credentials, API keys, endpoints. Written into the pages of the grimoire, that knowledge is one careless `git push` away from everybody else.

Your task is to build a configuration system based on **environment variables** and a `.env` file, so that the secrets live next to the laboratory and never inside the formulas.



Use the python-dotenv library to load environment variables from a .env file. The goal is to learn how .env files are used for configuration, not to write your own parser.

Mission

Create a program called `ex1/oracle.py` that:

- Loads its configuration from environment variables
- Uses a `.env` file for development settings

- Behaves differently in a development and in a production configuration
- Handles missing configuration properly, without crashing



You are free to choose how you demonstrate the development/production difference, but it must be visible in the output when testing your program.

Configuration

Your program should handle the following configuration variables:

- `MATRIX_MODE` - "development" or "production"
- `DATABASE_URL` - Connection string for data storage
- `API_KEY` - Secret key for external services
- `LOG_LEVEL` - Logging verbosity
- `ZION_ENDPOINT` - URL of the remote network



Never commit real secrets to version control! Your .env file must be listed in ex1/.gitignore , and you must be able to explain why.

Testing without any configuration:

```
$> python3 ex1/oracle.py
# Should show default values and warn about the missing configuration
```

Using a `.env` file:

```
$> cp ex1/.env.example ex1/.env
$> # Edit ex1/.env with your values
$> python3 ex1/oracle.py
# Should load the configuration from the .env file
```

Overriding through the environment:

```
$> MATRIX_MODE=production API_KEY=secret123 python3 ex1/oracle.py
# Should prefer the environment variables over the .env file
```

Example of expected output:

```
$> python3 ex1/oracle.py

ORACLE STATUS: Consulting the configuration...

Configuration loaded:
Mode: development
Database: Connected to local instance
API Access: Authenticated
Log Level: DEBUG
Remote Network: Online

Configuration security check:
[OK] No hardcoded secrets detected
[OK] .env file properly ignored by git
[OK] Production overrides available

The Oracle sees all configurations.
```



Why is an environment variable a better place for a secret than a constant in your source code? What is still *not* safe about it?

Chapter VI

Exercise 2: The Alembic

	Exercise2
alembic	
Files to Submit: `elements.py, alchemy/__init__.py, alchemy/elements.py, ft_alembic_0.py, ft_alembic_1.py`	
Authorized: `builtins, standard types, and only the modules you create in this project`	

The laboratory is sealed, stocked and configured. Time to write formulas.

Every alchemist starts with simple ingredients in their alembic. But some ingredients sit close at hand, and others live deep inside a cupboard. This exercise is about the `import` mechanisms that reach each of them, and about the gateway that decides what a package shows to the outside world.

First, build the components of the laboratory architecture.

elements.py (at the root of the repository) will contain:

- A function `create_fire()` that just returns the string "Fire element created".
- A function `create_water()` that just returns the string "Water element created".

alchemy/elements.py will contain:

- A function `create_earth()` that just returns the string "Earth element created".
- A function `create_air()` that just returns the string "Air element created".

alchemy/___init___.py will contain:

- Whatever is needed to expose `create_air` through the `alchemy` package interface, and **nothing** that exposes `create_earth` . See `ft_alembic_1.py` .

Now test that architecture with two scripts, each one reaching the elements a different way:

- `ft_alembic_0.py` will use the `import ...` structure to access `elements.py` at the root, and then create fire.
- `ft_alembic_1.py` will use `import alchemy` to go through the package interface, and then create air. It will then try to call `create_earth()` through that same interface: this must **fail**, even though `alchemy/elements.py` defines that very function. You may catch the exception or not, this is only for pedagogical purposes. A `mypy` error is expected here, on purpose.



You may face flake8 or mypy errors on your __init__.py file. Follow the usual conventions in that situation.

Example:

```
$> python3 ft_alembic_0.py
=== Alembic 0 ===
Using: 'import ...' structure to access elements.py
Testing create_fire: Fire element created

$> python3 ft_alembic_1.py
=== Alembic 1 ===
Accessing the alchemy module using 'import alchemy'
Testing create_air: Air element created
Now show that not all functions can be reached
This will raise an exception!
Testing the hidden create_earth: Traceback (most recent call last):
  File "/tmp/Python/module-05-bis/ft_alembic_1.py", line 12, in <module>
    print(f"{alchemy.create_earth()}")
          ^^^^^^^^^^^^^^^^^^^^^
AttributeError: module 'alchemy' has no attribute 'create_earth'. Did you mean: 'create_air'?
```



alchemy/elements.py defines create_earth, yet ft_alembic_1.py cannot reach it through import alchemy. Nothing was hidden, nothing was made private. So what exactly did __init__.py change?

Chapter VII

Exercise 3: Distillation and Transmutation

	Exercise3
transmutation	
Files to Submit: `alchemy/potions.py, alchemy/transmutation/ as a package, ft_distillation_0.py, ft_transmutation_0.py`	
Authorized: `builtins, standard types, and only the modules you create in this project`	

Master the art of the nested import: summon distant code from distant files, across modules and down into a subpackage.

Distillation

Improve the laboratory architecture with a new component.

alchemy/potions.py will contain:

- A function `healing_potion()` that returns the string "Healing potion brewed with '[created earth element]' and '[created air element]"'.
- A function `strength_potion()` that returns the string "Strength potion brewed with '[created fire element]' and '[created water element]"'.
- Whatever else is needed to reach the four fundamental elements. Note that two of them live outside the package.

Check these new formulas with one script:

- `ft_distillation_0.py` will use the `from ... import ...` structure to access `alchemy/potions.py` directly, then brew both the healing and the strength potion.

Example:

```
$> python3 ft_distillation_0.py
=== Distillation 0 ===
Direct access to alchemy/potions.py
Testing strength_potion: Strength potion brewed with 'Fire element created' and 'Water element created'
Testing healing_potion: Healing potion brewed with 'Earth element created' and 'Air element created'
```

The Great Transmutation

Now go one level deeper, into a subpackage, and settle the ancient debate between absolute and relative imports.

alchemy/transmutation/recipes.py will contain:

- A function `lead_to_gold()` that returns the string "Recipe transmuting Lead to Gold: brew '[created air]' and '[created strength potion]' mixed with '[created fire]"'.
- At least one absolute import and one relative import.

Create one script that reaches that recipe:

- `ft_transmutation_0.py` will use the `import ...` structure to access `alchemy/transmutation/recipes.py` directly, and then transmute lead into gold.

Example:

```
$> python3 ft_transmutation_0.py
=== Transmutation 0 ===
Using file alchemy/transmutation/recipes.py directly
Testing lead to gold: Recipe transmuting Lead to Gold: brew 'Air element created' and 'Strength potion
    brewed with 'Fire element created' and 'Water element created'' mixed with 'Fire element created'
```



When should an alchemist use an absolute pathway, and when a relative one?

Chapter VIII

Exercise 4: Avoid the Explosion

	Exercise4
grimoire	
Files to Submit: `alchemy/grimoire/ as a package, ft_kaboom_0.py, ft_kaboom_1.py`	
Authorized: `builtins, standard types, and only the modules you create in this project`	

The final mystery, and the one that destroys laboratories: the **circular dependency**, which occurs when two modules try to summon each other in an endless loop.

You will write the same feature twice: once so that it works, and once so that it explodes. The point is that the difference between the two is a single design decision, not a single line of syntax.

The light grimoire, which survives

alchemy/grimoire/light_spellbook.py will contain:

- A function `light_spell_allowed_ingredients()` that returns the list of ingredients allowed for light magic: “earth”, “air”, “fire”, “water”.
- A function `light_spell_record(spell_name: str, ingredients: str)` that returns a string stating whether the spell was recorded or rejected. That decision comes from the validator below.

alchemy/grimoire/light_validator.py will contain:

- A function `validate_ingredients(ingredients: str)` that returns a string containing the ingredients and the “VALID” or “INVALID” keyword. The ingredients are valid if they include at least one of the ingredients allowed by the spellbook, case insensitive.

Notice the shape of the problem: the spellbook needs the validator to decide, and the validator needs the spellbook to know what is allowed. Each file needs the other. There are several ways to keep that from exploding; pick one, and be ready to explain the alternatives during the evaluation.

The dark grimoire, which does not

Now write `dark_spellbook.py` and `dark_validator.py` as the dark counterparts, with `dark_`-prefixed function names, and dark ingredients: "bats", "frogs", "arsenic", "eyeball".

This time, write the mutual dependency the naive way, at the top of both files, and leave it broken. This pair is a demonstration of the failure, so it must genuinely fail.



Do not copy the light files and rename the functions. Write the dark pair deliberately, so that the import cycle is the thing you chose to change. That contrast is the whole exercise.

Finally, test a few spells:

- `ft_kaboom_0.py` will access the `grimoire` package and record a light spell, flawlessly.
- `ft_kaboom_1.py` will reach `dark_spellbook.py` directly and try to record a dark spell. This must fail and raise an exception, indicating that your laboratory has just exploded. You may catch it or not; it is only for pedagogical purposes.

Example:

```
$> python3 ft_kaboom_0.py
=== Kaboom 0 ===
Using grimoire module directly
Testing record light spell: Spell recorded: Fantasy (Earth, wind and fire - VALID)

$> python3 ft_kaboom_1.py
=== Kaboom 1 ===
Access to alchemy/grimoire/dark_spellbook.py directly
Test import now - THIS WILL RAISE AN UNCAUGHT EXCEPTION
Traceback (most recent call last):
  File "/tmp/Python/module-05-bis/ft_kaboom_1.py", line 7, in <module>
    from alchemy.grimoire.dark_spellbook import dark_spell_record
  File "/tmp/Python/module-05-bis/alchemy/grimoire/dark_spellbook.py", line 2, in <module>
    from .dark_validator import validate_ingredients
  File "/tmp/Python/module-05-bis/alchemy/grimoire/dark_validator.py", line 2, in <module>
    from .dark_spellbook import dark_spell_allowed_ingredients
ImportError: cannot import name 'dark_spell_allowed_ingredients' from partially initialized module '
    alchemy.grimoire.dark_spellbook' (most likely due to a circular import) (/tmp/Python/module-05-bis/
    alchemy/grimoire/dark_spellbook.py)
```



Both grimoires describe two files that need each other. Why does only one of them explode?

Chapter IX

Submission Instructions

Turn in your assignment in your `Git` repository, as usual. Only the work inside your repository will be evaluated during the defense.

Place the expected file tree at the root of your Git repository, and do not submit any virtual environment or `.env` file.



During evaluation, you may be asked to seal a fresh laboratory from scratch, to explain any import mechanism, to change what your package exposes, or to break and repair a circular dependency. Make sure you understand the mysteries, not just their implementation.